P1815R1: Translation-unit-local entities

Audience: CWG
S. Davis Herring <herring@lanl.gov>
Los Alamos National Laboratory
November 21, 2019

History

r1:

- Added example and fixed existing example
- Added NB comment reference
- Moved deduction guide wording to correct place
- Restored basic permission for `static` in header units
- Made more template specializations TU-local
- Clarified type specifiers
- Clarified TU-locality and usage of anonymous types

Introduction

This paper is a replacement for P1498R1 that includes the wording promised for it and thereby addresses US35. While its general principle of forbidding exposure of internal-linkage constructs to other translation units is followed, this paper presents rules that differ for consistency, especially for templates.

The function to which a dependent name refers cannot, of course, be determined merely from the template declaration that contains it. P1347R1 proposed forbidding such a call that selects an internal-linkage function via overload resolution. Such overload resolution can in general require undesirably extensive analysis of otherwise-local declarations, so P1498R1 goes further and (with a single exception for certain constants) forbids any use of such a name in a context where code generation might occur for another translation unit. (This includes, for example, the return type of a non-template function with more than internal linkage; while it need not be mangled, it is accessible via `decltype` anywhere the function is.) We can consider such overload sets to be *poisoned* by their mere inclusion of an unusable candidate. The unusable candidate might be a template; templates whose declarations involve internal entities must themselves be internal to avoid needing to perform template argument deduction for them (even if the resulting specialization might not involve any internal types). (As a reminder, `export` is irrelevant here because the client translation unit might be in the same module.)

However, ADL considers a set of functions and function templates which itself cannot be known until instantiation. It is therefore impossible to reliably preclude poisoned overload sets by any reasonable restriction on (non-internal) templates.

This paper therefore merely proscribes the external use of internal entities and of poisoned overload sets, and does so for the definition of a template only during instantiation. Even a template that unambiguously calls an internal function can have non-internal linkage so long as the only specializations used in other translation units are those generated by an explicit specialization or instantiation. (Implicit instantiation of a template does not preclude other implicit instantiations and so does not allow usage from other translation units.)

It would of course be possible to discard certain overload resolution candidates before deciding whether an overload set is poisoned. Most simply, any that do not support the number of arguments given in the call could be excluded. More sophisticated analysis might exclude functions or function templates with parameter types not convertible from a (non-dependent) argument type, as well as those whose constraints are known not to be met for any instantiation. Preferring future flexibility and compatibility over invention, this paper simply poisons overload sets unconditionally.

Wording

Relative to N4835.

Change [basic.lookup.argdep]/5:

- [*Example*:
  Translation unit #1:

```
export module M;
namespace R {
  export struct X ;
  export void f(X);
}
namespace S {
  export void f(X, X);
}
```

  Translation unit #2:

```
export module N;
import M;
export R::X make();
namespace R { static int g(X); }
export template<typename T, typename U> void apply(T t, U u) {
  f(t, u);
  g(t);
}
```

Translation unit #3:

```
module Q;
import N;
namespace S {
  struct Z { template operator T(); };
}
void test() {
  auto x = make();              // OK, decltype(x) is R::X in module M
  R::f(x);                      // ill-formed: R and R::f are not visible here
  f(x);                         // OK, calls R::f from interface of M
  f(x, S::Z());                 // ill-formed: S::f in module M not considered
                                // even though S is an associated namespace
  apply(x, S::Z());             // ~~OK,~~ill-formed: S::f is visible in instantiation context, ~~and~~but
                                // R::g ~~is visible even though it~~ has internal linkage and cannot be used outside TU #2
}
```

— *end example*]

Append paragraphs to [basic.link]:

A declaration *D names* an entity *E* if

1. *D* contains a *lambda-expression* whose closure type is *E*, or
2. *E* is not a function or function template and *D* contains an *id-expression*, *type-name*, *decltype-specifier*, *placeholder-type-specifier*, *template-name*, or *concept-name* denoting *E*, or
3. *E* is a function or function template that is named by an expression ([basic.def.odr]) appearing in *D*, or
4. *E* is a function or function template declared in one translation unit, *D* appears in another translation unit, and *D* contains an *id-expression* or *template-name* ([dcl.type.class.deduct]) that refers to a set of overloads that contains *E*.

A declaration instantiated for a template specialization ([temp.spec]) appears at the point of instantiation of the specialization ([temp.point]). [*Note*: Non-dependent names in such a declaration do not refer to a set of overloads ([temp.nondep]). — *end note*]

A declaration is an *exposure* if it names a TU-local entity (defined below), ignoring

1. the *function-body* for a non-inline function (but not the deduced return type for a (possibly instantiated) definition of a function with a declared return type that uses a placeholder type ([dcl.spec.auto])),
2. the *initializer* for a variable (but not the variable's type),
3. friend declarations in a class definition, and
4. any reference to a non-volatile const object with internal or no linkage initialized with a constant expression that is not an odr-use ([basic.def.odr]).

An entity is *TU-local* if it is

1. a type, function, variable, or template that
    1. has a name with internal linkage, or
    2. does not have a name with linkage and is declared, or introduced by a *lambda-expression*, within the definition of a TU-local entity, or
2. a type with no name introduced by a *defining-type-specifier* that is used to declare only TU-local entities, or
3. a specialization of a TU-local template, or
4. a specialization of a template with any TU-local template argument, or
5. a specialization of a template whose (possibly instantiated) declaration is an exposure. [*Note*: The specialization might have been implicitly or explicitly instantiated. — *end note*]

If a (possibly instantiated) declaration of, or a deduction guide for, a non-TU-local entity in a module interface unit ([module.unit]), module partition, or header unit ([module.import]) is an exposure, the program is ill-formed. Such a declaration in any other translation unit is deprecated ([depr.local]).

If a (possibly instantiated) declaration that appears in one translation unit names a TU-local entity declared in another translation unit, the program is ill-formed.

[*Example*:
Translation unit #1:

```
export module A;
static void f();
inline void it() {f();} // error: is an exposure of f
static inline void its() {f();} // OK
template<int> void g() {its();} // OK
```

```
template void g<0>();

static auto x=[]{f();};  // OK
auto x2=x;               // error: the closure type is TU-local
int y=([]{f();}(),0);    // error: the closure type is not TU-local
int y2=(x,0);            // OK

namespace N {
  struct A {};
  void adl(A);
  static void adl(int);
}
void adl(double);

inline void h(auto x) {adl(x);} // OK, but a specialization might be an exposure
```

Translation unit #2:

```
module A;
void other() {
  g<0>();  // OK: specialization is explicitly instantiated
  g<1>();  // error: instantiation uses TU-local its
  h(N::A{});  // error: overload set contains TU-local N::adl(int)
  h(0);  // OK: calls adl(double)
}
```

— *end example*]

Change [dcl.inline]/2:

- A function declaration ([dcl.fct], [class.mfct], [class.friend]) with an `inline` specifier declares an *inline function*. The inline specifier indicates to the implementation that inline substitution of the function body at the point of call is to be preferred to the usual function call mechanism. An implementation is not required to perform this inline substitution at the point of call; however, even if this inline substitution is omitted, the other rules for inline functions specified in this subclause shall still be respected. [*Note*: An inline function can use a name with internal linkage only if it is declared with internal linkage. — *end note*]

Change [dcl.inline]/7:

- An exported inline function or variable shall be defined in the translation unit containing its exported declaration, outside the *private-module-fragment* (if any). [*Note*: ~~There is no restriction on the linkage (or absence thereof) of entities that the function body of an exported inline function can reference.~~ A constexpr function ([dcl.constexpr]) is implicitly inline. — *end note*]

Change [dcl.spec.auto]/10:

- An exported function with a declared return type that uses a placeholder type shall be defined in the translation unit containing its exported declaration, outside the *private-module-fragment* (if any). [*Note*: T~~here is no restriction on the linkage of t~~he deduced return type cannot have a name with internal linkage. — *end note*]

Change [module.import]/5:

- […][*Note*: A *module-import-declaration* nominating a *header-name* is also recognized by the preprocessor, and results in macros defined at the end of phase 4 of translation of the header unit being made visible as described in [cpp.import]. — *end note*] A declaration of a name with internal linkage is permitted within a header unit despite all declarations being implicitly exported ([module.interface]). ~~If such a declaration declares an entity that is odr-used outside the header unit, or by a template instantiation whose point of instantiation is outside the header unit, the program is ill-formed.~~

Add a subclause before [depr.impldec]:

Non-local use of TU-local entities [depr.local]

A declaration of a non-TU-local entity that is an exposure ([basic.link]) is deprecated. [*Note*: Such a declaration in an importable translation unit is ill-formed. — *end note*] [*Example*:

```
namespace {
  struct A {
    void f();
  };
}
A h();                      // deprecated: not internal linkage
inline void g() {A().f();}  // deprecated: inline and not internal linkage
```

— *end example*]